S-Imply: Back Implication Prediction in ATPG using Multi-Path Attention

1st Chinthana Wimalasuriya
School of Electrical, Computer and Biomedical Engineering
Southern Illinois University
Carbondale, IL, USA
chinthana.w@siu.edu

2nd Spyros Tragoudas
School of Electrical, Computer and Biomedical Engineering
Southern Illinois University
Carbondale, IL, USA
spyros@siu.edu

***Abstract*—Automatic Test Pattern Generation (ATPG) relies heavily on iterative branch-and-bound techniques to justify structural fault conditions within digital logic circuits. Reconvergent fanout topologies present a persistent challenge, engendering structural logic conflicts that necessitate exhaustive backtracking. We propose S-Imply, an end-to-end differentiable framework for back implication prediction combining structural and functional embeddings with a Multi-Path Transformer architecture. Our approach explicitly models the local re-convergent regions to dynamically resolve justification constraints across interacting branches without relying on generic sparse rewards. Employing a policy-gradient-style curriculum utilizing the Gumbel-Softmax estimator, the model achieves a deterministic, learned proxy of Boolean Satisfiability (SAT) over subset circuit topologies.**

***Index Terms*—ATPG, Back Implication, Multi-Path Transformer, Electronic Design Automation, Reconvergent Fanouts**

I. Introduction

Modern Very Large Scale Integration (VLSI) testing mandates rigorous and highly efficient Automatic Test Pattern Generation (ATPG) techniques. Algorithms such as the Path-Oriented Decision Making (PODEM) algorithm [1] remain foundational paradigms. However, as circuit density scaling compounds, topological phenomena such as reconvergent fanouts, where logic signals branch from a common ancestral node (the stem) and converge upon a target node, induce significant algorithmic instability [2]. When attempting to assign and justify Boolean logic values, existing solvers commonly encounter logical contradictions (conflicts) isolated deep within these reconvergent topologies, triggering computationally demanding backtracking cascades.

Addressing these topologies inherently resembles subset Boolean Satisfiability (SAT) [3], prompting the exploration of machine learning heuristics to bypass the combinatorial explosion. While classical deep learning struggles with deterministic Boolean constraint compliance, recent advances in Graph Neural Networks (GNNs) offer high-fidelity structural logic embeddings (e.g., DeepGate) [4]. Yet, monolithic generative approaches frequently fail to resolve interdependent multi-path constraints.

In this paper, we introduce S-Imply (Structural Implication), a differentiable framework specifically architected to target and predict logically consistent Boolean assignments for localized reconvergent subsets. Instead of sparse Reinforcement Learning (RL) rewards, we enforce logical constraint adherence via an end-to-end differentiable loss framework mapping edge-wise gate satisfiability directly back to policy gradients via the Gumbel-Softmax estimator [5]. S-Imply acts as an AI-guided heuristic oracle intimately integrated into a novel recursive structure solver that guarantees dynamic path validation bounding for complex ATPG environments.

II. Related Work

Rigorous testing methodologies are paramount to ensure the functional correctness of modern integrated circuits. Automatic Test Pattern Generation (ATPG) generates deterministic input vectors to activate potential faults and propagate their effects to observable outputs [6]. Because this process is proven to be NP-complete, classical exact solvers are frequently overwhelmed by the combinatorial explosion inherent to high-density modern designs [7], [8], [9]. Consequently, the evolution of ATPG has transitioned toward integrating advanced machine learning (ML), particularly Graph Neural Networks (GNNs) and Transformers, to natively resolve complex Boolean constraints [10], [11].

A. Classical Algorithms and the Reconvergent Bottleneck

Systematic ATPG began with Roth's D-Algorithm, which formalized fault effect representations but suffered from massive, unmanageable decision trees [8], [12]. The PODEM algorithm optimized this by restricting the search space to Primary Inputs (PIs), capping the decision tree depth [8]. The FAN algorithm further accelerated generation using heuristics like "unique sensitization paths" and deferred justification of "headlines" [8], [12]. Alternatively, SAT-based ATPG translates the netlist and fault conditions into a Boolean Satisfiability (SAT) instance, highly effective for identifying untestable faults and verifying complex scan networks [13], [16].

Despite these heuristics, classical engines are severely hindered by *reconvergent fanouts*, structures where a signal splits at a stem and reconverges downstream. Backtracing across these branches frequently generates structural logic conflicts requiring catastrophic backtracking [12], [15].

B. Machine Learning Interventions

1) Shallow Heuristics and ANNs: Early ML integrations utilized shallow Artificial Neural Networks (ANNs) as predictive oracles to recommend optimal backtrace directions, reducing decision tree depth [7], [18]. However, traditional MLPs expect fixed-dimensional data, destroying the non-Euclidean topological connectivity of digital circuits [19], [15]. Furthermore, querying these models often introduced inference latency that outweighed the computational time saved.

2) GNNs for Circuit Representation: Graph Neural Networks (GNNs) natively process circuit topologies, producing vector embeddings that encapsulate structural connectivity and functional behavior [10], [19].

- **DeepGate1 & 2**: The original DeepGate applied message-passing GNNs to And-Inverter Graphs (AIGs) using logic-1 probabilities as supervision [10], [19]. DeepGate2 drastically improved this by training on pairwise truth-table differences (Hamming distance) and using an efficient single-round forward architecture, achieving a 16.4x speedup [19], [15], [21].
- **DeepGate3**: To combat "over-smoothing" in massive circuits, DeepGate3 employs a hybrid GNN-Transformer framework with a Pooling Transformer (PT) block to capture long-range gate dependencies, achieving a 50.4% runtime reduction in SAT solving [10], [15].
- **GANA**: This model integrates topological netlist data with geometrical layout information for automated annotation and knowledge transfer [15], [20].

3) Accelerated ATPG and Smart Generation: Specialized architectures couple neural representations with parallelized execution:

- **FPGNN-ATPG**: This framework fuses deep learning with multi-core parallelization. A GNN preemptively classifies fault detectability to prune the fault list. It dynamically routes simple faults to a FAN engine and complex faults to a SAT solver, achieving a 7.56x speedup over commercial tools while maintaining 100% coverage [15], [23].
- **InF-ATPG**: This Reinforcement Learning (RL) framework models ATPG as a Markov Decision Process. To overcome massive action spaces, it partitions circuits into Fanout-Free Regions (FFRs). By treating entire FFRs as macro-actions, and using a Quality-Value GNN (QGNN) for state embedding, it reduces required backtracks by 55% compared to classical PODEM [8], [15].

C. Transformers and Differentiable Logic

1) Sequence-to-Sequence Logic Reasoning: Decoder-only Transformers can natively parse Conjunctive Normal Form (CNF) constraints and decide 3-SAT instances using Chain-of-Thought (CoT) prompting [14]. Models like DiLA and TG-SAT achieve up to 65x speedups on complex constraints [9], [24]. To prevent hallucinations, the **Circuit Transformer** utilizes a deterministically constrained decoding mechanism that actively simulates partial circuits, blocking any token that violates strict logical equivalence [11], [25].

2) Differentiable Logic for Reconvergent Fanouts: Discrete logic gates natively block gradient backpropagation. Researchers introduced the **Gumbel-Softmax estimator** to perform continuous relaxation, rendering discrete sampling differentiable [15], [26].

Leveraging this, the **S-Imply** architecture acts as a deterministic, learned SAT proxy specifically for Local Reconvergent Regions (LRRs). Utilizing a Multi-Path Transformer, it applies cross-attention between converging topological branches. Optimized via a multi-objective soft-compliance loss, S-Imply predicts conflict-free logic assignments within reconvergent fanouts, actively aborting infinite backward justification arrays significantly faster than classical trace enumeration [15].

D. Hardware Acceleration

The overhead of GNN and Transformer inference mandates hardware acceleration. Implementations heavily rely on highly parallelized GPUs to accelerate SAT evaluation [23], [28]. For edge deployment on Automatic Test Equipment (ATE), quantized neural models are synthesized into Field Programmable Gate Arrays (FPGAs) via High-Level Synthesis (HLS), ensuring deterministic, sub-20 millisecond response times required for real-time silicon reliability screening [27], [15].

III. Proposed Methodology

The objective of S-Imply is to establish an offline, trained policy distribution $\pi(a|s)$ capable of rapidly predicting optimal, conflict-free logic assignments (0 or 1) for all internal nodes comprising a selected reconvergent subset $S \subseteq G$, given graph $G = (V, E)$. The resulting logic assignment strictly adheres to three structural invariants: local gate consistency, reconvergence equivalence, and common structural ancestor equivalence (stem consistency).

A. Problem Formulation and Structure Identification

To systematically isolate sub-graphs, we draw upon the formal topological definitions established by Maamari and Rajski [2]. A circuit is modeled as a directed acyclic graph (DAG) $G = (V, E)$, where V represents logic gates and E represents signal nets.

1) Formal Definition of Local Reconvergent Regions: We defines a *Fanout Stem* v_s as any node with out-degree $d^+(v\_s) > 1$. A node v_r is a *Reconvergence Point* for v_s if there exist at least two vertex-disjoint paths P_1, P_2 originating at v_s and terminating at v_r. The Local Reconvergent Region (LRR), denoted $\mathcal{L}(v\_s, v\_r)$, is the sub-graph induced by the set of all nodes lying on any path from v_s to v_r.

2) Structural Bounding via Dominance: To ensure the LRR is "locally justifiable," we apply a dominance-bounded search. We say v_i dominates v_j if every path from v_j to any primary output (PO) passes through v_i. Identifying LRRs bounded by dominators ensures that predicted logic assignments do not create unforeseen constraints that propagate arbitrarily across the entire netlist.

3) Exit Line Indexing: A critical concept determining local autonomy is the presence of *Exit Lines*. An exit line $e \in \mathcal{L}$ is a node within the LRR that provides an input to a gate $g \in G \backslash \mathcal{L}$. Our identification algorithm prioritizes LRRs with minimal exit lines to maximize the decoupled solving efficiency. To ensure $O(1)$ lookup performance during deep recursive state validation, specifically critical for dense combinational circuits (e.g., ISCAS85 c6288), we index all identified Exit Lines into a localized *exit map* data structure. This map allows the solver to instantly determine if a predicted assignment at node $v \in \mathcal{L}$ has immediate implications on the global circuit state outside the neural domain.

B. Multi-Path Transformer Architecture

Rather than treating the local circuit region as an unstructured set of nodes, we explicitly model the distinct branches of a reconvergent structure. Our model employs a Multi-Path Transformer parameterized to simultaneously process independent parallel paths while facilitating global cross-branch attention.

1) Feature Representation and Multi-Modal Embeddings: For each node $v_i \in P_j$, a base feature vector $\mathbf{x}_i^{\text{base}} \in \mathbb{R}^{d_{\text{base}}}$ is assembled by concatenating structural and constraint-aware signals:

$$\mathbf{x}_i^{\text{base}} = [\mathbf{h}_{\text{struct}}(v_i) \parallel \mathbf{c}(v_i)] \tag{1}$$

where $\parallel$ denotes concatenation. This base vector is augmented on-the-fly with a learned gate-type embedding before the input projection:

$$\mathbf{x}_i = [\mathbf{x}_i^{\text{base}} \parallel \mathbf{e}_{\text{gate}}(t_i)] \in \mathbb{R}^{d_{\text{total}}} \tag{2}$$

1) **DeepGate Structural Embedding ($\mathbf{h}_{\text{struct}} \in \mathbb{R}^{128}$)**: Extracted from a pre-trained GNN (DeepGate) that encodes circuit topology by predicting logic-1 probabilities via message passing on And-Inverter Graphs [4]. This vector encapsulates both the global topological context and the functional behavioral signature of each node. Embeddings are cached per circuit in a `.deepgate_cache/` directory to avoid redundant inference.
2) **Logic Constraint Channels ($\mathbf{c} \in \mathbb{R}^4$)**: A four-dimensional one-hot vector encoding the ATPG five-valued implication state of each node drawn from {ZERO, ONE, XD, D, $\overline{D}$}. Three of these dimensions encode the constraint visible at inference time (`[unknown, zero, one]`), conditioned on partial PI assignments accumulated by PODEM backtracing.
3) **Gate Type Lexicon ($\mathbf{e}_{\text{gate}} \in \mathbb{R}^{64}$)**: A learned embedding for each discrete gate operator t_i, drawn from a vocabulary of 12 types covering `INPT`, `FROM`, `BUFF`, `NOT`, `AND`, `NAND`, `OR`, `NOR`, `XOR`, `XNOR` and two auxiliary types. This lexicon ensures the model distinguishes inversion boundaries from non-inverting propagation.

The base dimension $d_{\text{base}} = 132$ (128 structural + 4 constraint). After gate-type augmentation, $d_{\text{total}} = 196$, which is projected into the Transformer's model dimension $d_{\text{model}} = 512$ via a learned linear layer.

2) Shared Path and Interaction Encoders: Let the input matrix for path j be $X^{(j)} = [x_1^{(j)}, \ldots, x_{N_j}^{(j)}]^T \in \mathbb{R}^{N_j \times d}$. We compute an intermediate node representation $H^{(j)}$ through a shared path encoder:

$$H^{(j)} = \text{TransformerEncoder}(X^{(j)}; \Theta\_shared) \tag{3}$$

This operation captures localized sequential signal flows, such as chained buffer propagation paths or progressive gate constraint cascades, independently for each branch. By sharing Θ_shared across all paths, the model learns a universal topological language for signal propagation.

3) Interaction Transformer and Multi-Head Cross-Attention: To capture the complex interdependencies between reconverging branches, we extract a per-path summary vector $z^{(j)}$ by selecting the representation of the terminal (reconvergence-point) node in each path:

$$z^{(j)} = H_{N_j}^{(j)} \tag{4}$$

where N_j is the index of the last valid (non-padding) token in path j. This choice is deliberate: the terminal node is the reconvergent node itself, so its encoding in $H^{(j)}$ already integrates all upstream sequential constraints along branch j after the shared encoder pass. These summaries represent the "logical state" of each branch at the point of convergence. The Interaction Transformer maps these summaries through $N\_interact = 3$ layers of self-attention:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \tag{5}$$

where Q, K, V are linear projections of the summary matrix $\mathbf{Z} = [z^{(1)}, z^{(2)}]^T$. This allows the representation of path P_1 to be contextualized by the requirements of P_2.

4) Cross-Attention Polling and Hierarchical Prediction: The independent node elements $H^{(j)}$ reconcile against the interactive constraints $\hat{z}$ via a polling mechanism. Every node vector $h_i^{(j)}$ acts as a query in a cross-attention layer:

$$\text{Context}_i^{(j)} = \sum_{m=1}^{M} \text{Head}_m(h_i^{(j)}, \hat{Z}, \hat{Z}) \tag{6}$$

where $M = 4$ is the number of attention heads. The cross-attention mechanism effectively allows each node to "poll" the global interaction summaries for consistency constraints. The updated state incorporates both the cross-attention result and the path's own interaction-aware summary $\hat{z}^{(j)}$ via a triple residual:

$$y_i^{(j)} = \text{LayerNorm}\Big(h_i^{(j)} + \text{Context}_i^{(j)} + \hat{z}^{(j)}\Big) \tag{7}$$

This final state $y_i^{(j)}$ is projected through parallel classification heads:

- **Logic Polarity Head**: A linear projection $W\_p \in \mathbb{R}^{d\_model \times 2}$ maps each $y_i^{(j)}$ to binary logits $P(\text{val}(v_i) = c) \in \mathbb{R}^2$.
- **Regional Solvability Head**: $S\_LRR \in \mathbb{R}^2$ is computed by applying a linear projection to the mean of

the interaction-aware path summaries $\hat{z}^{(j)}$: $S\_LRR = W\_s \cdot \frac{1}{|P|}\sum_j \hat{z}^{(j)}$. This provides a circuit-level solvability estimate grounded in inter-branch consistency.

C. Differentiable Logical Training Framework

A fundamental challenge in training neural models over Boolean circuits is that $\partial v/\partial u = 0$ almost everywhere for discrete gate functions, prohibiting direct gradient backpropagation. This section describes how the Gumbel-Softmax estimator is used to construct a fully differentiable surrogate, and defines the multi-objective loss that enforces circuit-consistent predictions.

1) Gradient Reparameterization: Let $l_v \in \mathbb{R}^2$ be the logits for node v. During training, differentiable soft assignments a_v are sampled using the Gumbel-Softmax estimator [5] with a straight-through estimator:

$$a_v = \frac{\exp((l_{v,1} + g_{v,1})/\tau)}{\sum_{j=0}^{1} \exp((l_{v,j} + g_{v,j})/\tau)} \tag{8}$$

where $g_{v,j} \sim \text{Gumbel}(0, 1)$. With `hard=True`, the forward pass produces discrete one-hot values while the backward pass propagates gradients through the continuous Gumbel-Softmax distribution, allowing end-to-end differentiation through discrete gate-logic checks. The temperature τ is annealed per epoch as described in Section III-C4.

2) Multi-Objective Logic Loss: The model is optimized via a weighted composite loss function:

$$\mathcal{L}_{\text{total}} = \lambda_e \mathcal{L}_{\text{edge}} + \lambda_r \mathcal{L}_{\text{reconv}} + \lambda_c \mathcal{L}_{\text{const}} + \lambda_s \mathcal{L}_{\text{solv}} + \lambda_{\text{sn}} \mathcal{L}_{\text{shared}} - \beta_H \mathcal{H} \tag{9}$$

The total loss is computed as the sum of all active terms; default scalar weights are $\lambda_e = 1.0$, $\lambda_r = 0.5$ (scaled to balance against $\mathcal{L}_{\text{edge}}$), $\lambda_c = 1.0$, $\lambda_s = 1.0$, $\lambda_{\text{sn}} = 1.0$, and entropy coefficient $\beta_H = 0.01$. The solvability loss applies an asymmetric class weight of $w_{\text{UNSAT}} = 10$, $w_{\text{SAT}} = 1$ to address severe label imbalance.

Soft Edge Consistency ($\mathcal{L}\_edge$): Rather than enforcing hard gate truth tables, we impose a soft differentiable penalty on gate-logic violations using the Gumbel-Softmax probabilities $a\_v \in [0, 1]$. For a single-input gate, violations are measured exactly; for multi-input AND/OR families, a hinge formulation penalizes only infeasible output values, since controlling inputs can produce valid logic even in partial assignments:

- *NOT*: $\omega\_g(a\_v - (1 - a\_u))^2$
- *BUFF*: $\omega\_g(a\_v - a\_u)^2$
- *AND*: $\omega\_g\,\text{ReLU}(a\_v - a\_u)^2$ (output cannot exceed any input)
- *NAND*: $\omega\_g\,\text{ReLU}((1 - a\_u) - a\_v)^2$
- *OR*: $\omega\_g\,\text{ReLU}(a\_u - a\_v)^2$ (output cannot fall below any input)
- *NOR*: $\omega\_g\,\text{ReLU}(a\_v - (1 - a\_u))^2$
- *XOR (2-input)*: $\omega\_g(a\_u\_1(1 - a\_u\_2) + a\_u\_2(1 - a\_u\_1) - a\_v)^2$

Gate-type weights are $\omega\_g = 2.0$ for *rigid gates* (NOT, BUFF) where a single input deterministically fixes the output, and $\omega\_g = 1.0$ for *controlling-value gates* (AND, NAND, OR, NOR). The total edge loss is the mean across all valid in-path edges.

Reconvergence Consensus ($\mathcal{L}\_reconv$): At the shared reconvergent node v_r, each path P_j in the LRR produces an independent logit vector $l\_v\_r^{(j)} \in \mathbb{R}^2$. We enforce cross-path agreement by penalizing deviation from the mean terminus logit:

$$\bar{l}\_v\_r = \frac{1}{|P|}\sum_j l\_v\_r^{(j)}, \qquad \mathcal{L}\_reconv = \frac{1}{|P|}\sum_j \left\| l\_v\_r^{(j)} - \bar{l}\_v\_r \right\|^2 \tag{10}$$

Operating on logits rather than discrete samples preserves full gradient flow through the penalty. The loss is scaled by $\lambda_r = 0.5$ to balance its magnitude relative to the edge loss.

Shared-Node Consistency ($\mathcal{L}_{\text{shared}}$): A circuit node may appear on multiple reconvergent paths simultaneously (i.e., it is a shared stem). An unconstrained model may assign contradictory values to the same node across different paths. For every node v that appears at $K \geq 2$ valid positions within a batch sample, the variance of the predicted probability $P(\text{val}(v) = 1)$ across those positions is penalized:

$$\mathcal{L}_{\text{shared}} = \frac{1}{|\mathcal{S}|}\sum_{v \in \mathcal{S}} \frac{1}{K_v}\sum_{k=1}^{K_v} \left(p_v^{(k)} - \bar{p}_v \right)^2 \tag{11}$$

where $\mathcal{S}$ is the set of nodes appearing on ≥ 2 paths, K_v is the count of such appearances, $p_v^{(k)}$ is the Gumbel-Softmax probability at the k-th occurrence, and $\bar{p}_v$ is the mean over all occurrences.

Constraint Supervision ($\mathcal{L}_{\text{const}}$): A subset of nodes are designated as *anchor* nodes with circuit-verified Boolean values derived from real Primary Input assignments accumulated by the hierarchical solver. For these nodes, a cross-entropy supervision loss is applied:

$$\mathcal{L}_{\text{const}} = -\frac{1}{|\mathcal{A}|}\sum_{(v,c) \in \mathcal{A}} \log P(\text{val}(v) = c) \tag{12}$$

where $\mathcal{A}$ denotes the set of anchor node–value pairs. Anchor supervision is restricted to SAT-labeled samples (solvability label $= 0$) to avoid imposing contradictory gradients on structurally infeasible assignments.

Entropy Regularization ($\mathcal{H}$): To prevent premature prediction collapse, we add an entropy bonus:

$$\mathcal{H} = \frac{1}{|V|}\sum_v H(\text{softmax}(l_v)) \tag{13}$$

which penalizes overconfident distributions and sustains exploratory capacity across the curriculum.

3) Constraint Curriculum and Anchor Injection: Training data is generated by `build_fault_dataset.py`, which executes the hierarchical solver over each fault in the ISCAS benchmark suite and records each encountered reconvergent pair as a separate training sample. The constraint density within each sample increases naturally with the solver's execution order: the first pair solved for any given fault carries only

the terminus constraint (target node v_r), while each subsequent pair inherits all path-node assignments accumulated from previously solved pairs. This produces a training distribution whose constraint density grows from near-zero to heavily constrained, precisely matching the inference distribution without requiring any stochastic ramp schedule at training time.

All constraint values are circuit-consistent: they originate from actual Boolean simulations under valid Primary Input assignments, which ensures that every supervised gradient signal is satisfiable under the true gate functions. No randomly generated or potentially contradictory constraint values are introduced.

4) Training Optimization Strategies: The training loop in `src/ml/train.py` employs several strategies for numerical stability and efficient throughput.

Gumbel Temperature Annealing: The temperature τ is decayed once per epoch according to:

$$\tau_t = \max(0.1,\ \tau_0 \cdot 0.99^{\,t-1}) \tag{14}$$

with $\tau_0 = 1.0$. During early epochs the soft distribution encourages exploration; as training progresses and $\tau \to 0.1$, the straight-through estimator produces near-discrete outputs that increasingly resemble Boolean logic. At evaluation time, a fixed $\tau = 0.1$ is used.

Mixed Precision and Gradient Clipping: All forward and backward passes are wrapped in `torch.amp.autocast` when AMP is enabled, reducing GPU memory consumption. Gradient norms are clipped to a maximum of 1.0 before each optimizer step to prevent AMP-induced gradient explosions.

Path Shuffling: Path order within each batch item is randomly permuted at every training iteration, preventing the model from learning positional biases between path 0 and path 1.

ShardBatchSampler: When preprocessed tensor shards are available, the `ShardBatchSampler` groups all indices belonging to the same shard into each batch. This allows the data loader to extract an entire batch via a single tensor-slice operation instead of one `clone()` call per sample, reducing CPU overhead by approximately $10\times$.

Learning Rate Schedule: An AdamW optimizer is used with initial learning rate $\eta = 10^{-4}$. A linear warmup is applied over the first $\min(3, \lfloor T/4 \rfloor)$ epochs, followed by cosine annealing to a minimum learning rate of 10^{-6} over the remaining epochs.

D. AI-PODEM: Neural-Augmented Justification

The core novelty of S-Imply lies in its integration with the backtrace engine of PODEM. The `AIBacktracer` class replaces the standard `simple_backtrace` function with a neural oracle that resolves reconvergent objectives hierarchically before delegating residual objectives back to the classical algorithm.

1) HierarchicalReconvSolver Algorithm: The `HierarchicalReconvSolver` is the algorithmic core of the AI-PODEM framework. Given a target gate v_t and required value ρ, it executes the following steps:

1) **Transitive Fan-in Collection**: A backward BFS is performed from v_t up to a depth of 20 levels to collect the transitive fan-in cone $\mathcal{C}(v_t)$.
2) **Reconvergent Pair Discovery**: Within $\mathcal{C}(v_t)$, all nodes with out-degree ≥ 2 (stems) are identified. For each stem s, a forward BFS from each direct fan-out branch tracks which branch first-indexes reach subsequent nodes; when a node is reached from ≥ 2 distinct branches, a reconvergent pair (s, r) is recorded.
3) **Priority Ordering**: Pairs are sorted by a composite cost

$$\text{cost}(p) = \big(\text{total\_path\_len}(p) + \text{dist}(p.\text{reconv},\ v_t),\ \text{total\_path\_len}(p)\big) \tag{15}$$

in ascending lexicographic order, so shorter, topologically closer pairs are resolved first. This ordering reflects the natural execution order in the training data and minimizes the number of constraints that must be inferred before reaching the target.
4) **Iterative Prediction**: Pairs are consumed in priority order. For each pair, the `ReconvPairPredictor` (`ModelPairPredictor` in inference) queries the Multi-Path Transformer with the current accumulated constraint dictionary. If the predictor returns a valid assignment, all predicted node values are merged into the running constraint set.
5) **Pair Cache Persistence**: To avoid redundant BFS on repeated calls for the same target (common in PODEM's backtrack-retry cycle), all discovered pairs are stored in a disk-backed pickle cache at `.reconv_cache/` alongside each `.bench` file.

2) AIBacktracer Integration: `AIBacktracer` wraps `HierarchicalReconvSolver` as a drop-in callable for PODEM's backtrace hook. On each invocation with a PODEM objective `Fault(gate_id, value)`:

1) **Structural Pre-check**: If no reconvergent pairs exist in the transitive fan-in of `gate_id` (cached from a prior call), the AI path is bypassed immediately and `simple_backtrace` is returned. This fast path eliminates neural inference overhead for fan-out-free regions and primary inputs.
2) **Constraint Extraction**: All Primary Inputs (PIs) currently assigned a definite value (`ZERO` or `ONE`) by PODEM's implication state are collected into a constraint dictionary $\mathcal{K}$.
3) **Solver Invocation**: `HierarchicalReconvSolver.solve(ga value,` $\mathcal{K}$`)` is called. The solver traverses the sorted pairs, calling `ModelPairPredictor.predict()` for each, accumulating a global assignment $\mathbf{a}$.
4) **Result Extraction**: From $\mathbf{a}$, the backtrace searches for a free PI (currently assigned `XD`). If one is found, it is returned as the next PODEM objective `Fault(pi_id, val)`. If no free PI exists but an unassigned internal node is present, `simple_backtrace` is delegated with that internal node as a sub-objective.
5) **Fallback**: If the solver returns no assignment (e.g., the predictor rejects all candidate assignments), or if

any exception occurs, `simple_backtrace` is invoked on the original objective. This ensures S-Imply never degrades coverage below classical PODEM.

The `ModelPairPredictor` additionally applies `post_process_logic_gates()`, a deterministic forward-propagation pass that corrects NOT and BUFF gate outputs along each path based on the current constraint state, enforcing those rigid gate rules before the assignment is returned to PODEM.

3) Confidence-Guided Search Retry: To recover from incorrect or sub-optimal predictions by the Multi-Path Transformer without abandoning the neural-augmented search entirely, S-Imply implements a confidence-guided retry mechanism. During recursive justification, the solver tracks each committed AI prediction on a decision stack, recording both the pair identifier and the minimum confidence score of the selected candidate assignment. If a global conflict is detected downstream (causing a justification failure), the solver does not backtrack exhaustively. Instead, it identifies the committed decision with the lowest confidence score in the stack, adds this pair to a forced-skip set, and triggers a retry. When a pair is in the forced-skip set, the solver bypasses the neural predictor for that region, instead delegating its justification to standard, deterministic gate-logic rules. This hybrid fallback loop allows the solver to recover from isolated marginal predictions, significantly improving robustness on deep reconvergent cones.

4) Complexity Analysis: Traditional PODEM justification in a circuit with N gates and D decision depth exhibits a worst-case complexity of $O(2^D)$. Reconvergent fanouts exacerbate this by forcing the branch-and-bound engine to explore multiple vertex-disjoint paths repeatedly. In contrast, S-Imply reduces the local decision complexity to $O(1)$ inference per LRR. While the pre-processing structure identification takes $O(V + E)$, the neural-assisted step reduces the effective backtracking depth D by a factor proportional to the number of LRRs resolved successfully. For a circuit with k identified LRRs, each of average logic depth Δd, the search space contracts from 2^D to $2^{D-\Delta d \cdot k}$, providing an exponential speedup in the limit of dense, deep reconvergent logic. Pair-topology results are persisted to disk after the first run, amortising the $O(V+E)$ BFS cost over all subsequent ATPG invocations on the same circuit.

IV. Experimental Results

In this section, we evaluate the performance of S-Imply against classical algorithms. First, we present a head-to-head comparison between AI-guided PODEM and classic PODEM on a subset of the ITC99 benchmark suite. Second, we evaluate the efficacy of the proposed model on a large pool of faults that were aborted by the classical ATPG tool Atalanta.

A. Evaluation on ITC99 Benchmarks

We evaluated S-Imply on the ITC99 $b17$ gate-level fault list using a deterministically selected 10% sample comprising 6,445 faults. We compare the AI-Guided PODEM against Classic PODEM under identical search constraints, utilizing a 30-second timeout and a limit of 5,000 backtracks.

On a representative head-to-head segment of 546 faults, Classic PODEM successfully detected 457 faults (83.7% coverage), while AI-Guided PODEM detected 444 faults (81.3% coverage). This corresponds to a relative coverage of 97.2% compared to the reference classic solver, confirming that S-Imply maintains high coverage while utilizing non-exhaustive heuristic search.

Table I summarizes the efficiency metrics for both solvers. S-Imply achieves a 2.4× overall speedup in total wall time. This acceleration is particularly pronounced on failed and untestable faults, where the classic solver exhausts the entire 30-second timeout, whereas AI-Guided PODEM fails fast (averaging 8.76 seconds) due to early identification of unsatisfiable assignments. Furthermore, on faults solved by both solvers, S-Imply reduces the total number of backtracks from 1,208 to 619, representing a 48.8% reduction.

Notably, 313 out of the 546 faults (57.3%) were resolved entirely in the structural pre-check phase. For these cases, S-Imply generated a circuit-consistent assignment in a single neural forward pass, achieving zero-backtrack, zero-PODEM-search direct detection.

TABLE I
ATPG Efficiency Comparison on ITC99 $b17$ Faults

Metric	AI-Guided PODEM	Classic PODEM
Faults Attempted	546	546
Faults Detected	444 (81.3%)	457 (83.7%)
Mean Time (Success)	0.97 s	1.21 s
Mean Time (Failure)	8.76 s	30.00 s
Total Wall Time	1,322 s	3,228 s
Total Backtracks (Success)	619	1,208

B. Resolution of Atalanta-Aborted Faults

To demonstrate the capabilities of S-Imply in resolving highly complex structural configurations, we constructed an aborted fault pool consisting of 19,755 unique faults from ITC99 and ISCAS circuits that the classical ATPG tool Atalanta aborted (i.e., failed to detect or prove redundant after reaching its maximum backtrack ceiling of 100,001 or 500,001).

We ran S-Imply over the entire pool of 19,755 aborted faults. The framework successfully resolved and generated test patterns for **12,787 faults**, achieving a **64.73% coverage** on this previously unsolvable pool.

To analyze the contribution of the reconvergence reasoning, we partition the pool into two cohorts:

1) **Reconvergent Cohort**: 9,887 faults containing reconvergent path pairs in the transitive fan-in of their target gate. S-Imply successfully detected **5,899 faults (59.66%)** in this cohort.
2) **Non-Reconvergent Cohort**: 9,868 faults with no reconvergent path pairs in the target gate's fan-in cone. S-Imply successfully detected **6,888 faults (69.80%)**

in this cohort by resolving downstream propagation conflicts.

On the 12,787 successful detections, Atalanta had accumulated a total of 1.287 billion representative abort backtracks (mean of 100,684 backtracks per fault). In contrast, S-Imply required a total of **only 41 search backtracks** across all successes (mean of 0.0032), yielding a search space reduction of **99.9999%**. Table II details the per-circuit performance across the benchmark suite.

TABLE II
S-IMPLY PERFORMANCE ON ATALANTA-ABORTED FAULT POOL

Circuit	Attempted Faults	Solved Faults	AI Coverage (%)
$b14$	43	42	97.67%
$b15$	649	632	97.38%
$b17$	1,718	1,607	93.54%
$b18$	1,776	1,310	73.76%
$b19$	14,630	8,529	58.30%
$b20$	218	214	98.17%
$b21$	120	120	100.00%
$b22$	312	303	97.12%
$c6288$	284	28	9.86%
Others	5	2	40.00%
Total	**19,755**	**12,787**	**64.73%**

V. CONCLUSION

We have presented S-Imply, a topological graph prediction approach to address the reconvergent fanout problem in ATPG environments. Leveraging multi-path attention models, the policy generates robust deterministic bounding structures scaling natively in multi-constraint validation regimes without relying on strict, fully-annotated logical datasets.

REFERENCES

[1] P. Goel, "An Implicit Enumeration Algorithm to Generate Tests for Combinational Logic Circuits," *IEEE Transactions on Computers*, vol. C-30, no. 3, pp. 215–222, Mar. 1981.

[2] B. Maamari and J. Rajski, "A Method of Fault Simulation Based on Stem Regions," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 9, no. 2, pp. 212–220, Feb. 1990.

[3] J. P. Marques-Silva and K. A. Sakallah, "GRASP: A Search Algorithm for Propositional Satisfiability," *IEEE Transactions on Computers*, vol. 48, no. 5, pp. 506–521, May 1999.

[4] M. Li, S. Khan, Z. Shi, N. Wang, H. Yu, and Q. Xu, "DeepGate: Learning Neural Representations of Logic Gates," in *Proc. 59th ACM/IEEE Design Automation Conf. (DAC)*, 2022, pp. 667–672.

[5] E. Jang, S. Gu, and B. Poole, "Categorical Reparameterization with Gumbel-Softmax," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2017.

[6] M. L. Bushnell and V. D. Agrawal, "Essentials of Electronic Testing for Digital, Memory and Mixed-Signal VLSI Circuits," Springer, 2000.

[7] S. Roy, S. K. Millican, and V. D. Agrawal, "Machine Intelligence for Efficient Test Pattern Generation," in *Proc. IEEE International Test Conference (ITC)*, 2020.

[8] B. Sun, R. Zhang, and Z. Xue, "InF-ATPG: Intelligent FFR-Driven ATPG with Advanced Circuit Representation Guided Reinforcement Learning," arXiv:2512.00079, Nov. 2025.

[9] P. Stephan, R. K. Brayton, and A. L. Sangiovanni-Vincentelli, "Combinational test generation using satisfiability," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 9, pp. 1167–1176, Sept. 1996.

[10] Z. Shi, Z. Zheng, S. Khan, J. Zhong, M. Li, and Q. Xu, "DeepGate3: Towards Scalable Circuit Representation Learning," in *Proc. 43rd IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2024, arXiv:2407.11095.

[11] X. Li, X. Li, L. Chen, X. Zhang, M. Yuan, and J. Wang, "Circuit Transformer: A Transformer That Preserves Logical Equivalence," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025, arXiv:2403.13838.

[12] H. Fujiwara and T. Shimono, "On the acceleration of test generation algorithms," *IEEE Transactions on Computers*, vol. C-32, no. 12, pp. 1137–1144, Dec. 1983.

[13] R. Baranowski, M. A. Kochte, and H.-J. Wunderlich, "Modeling, Verification and Pattern Generation for Reconfigurable Scan Networks," in *Proc. IEEE Int. Test Conf. (ITC)*, 2012.

[14] L. Pan et al., "Can Transformers Reason Logically? A Study in SAT Solving," in *Proc. 42nd Int. Conf. Machine Learning (ICML)*, 2025, arXiv:2410.07432.

[15] G. Huang *et al.*, "Machine Learning for Electronic Design Automation: A Survey," *ACM Transactions on Design Automation of Electronic Systems (TODAES)*, vol. 26, no. 5, pp. 1–46, 2021.

[16] R. Baranowski, M. A. Kochte, and H.-J. Wunderlich, "Reconfigurable Scan Networks: Modeling, Verification, and Optimal Pattern Generation," *ACM Trans. Design Automation of Electronic Systems (TODAES)*, vol. 20, no. 2, 2015.

[17] J. P. Roth, "Diagnosis of Automata Failures: A Calculus and a Method," *IBM Journal of Research and Development*, vol. 10, no. 4, pp. 278–291, 1966.

[18] W. Li, H. Lyu, S. Liang, T. Wang, P. Tian, and H. Li, "Intelligent Automatic Test Pattern Generation for Digital Circuits Based on Reinforcement Learning," in *Proc. 32nd IEEE Asian Test Symp. (ATS)*, Beijing, China, Oct. 2023.

[19] Z. Shi, H. Pan, C. Hu, and Z. Xie, "DeepGate2: Functionality-Aware Circuit Representation Learning," in *Proc. 42nd IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023, arXiv:2305.16373.

[20] K. Kunal *et al.*, "GANA: Graph Convolutional Network Based Automated Netlist Annotation for Analog Circuits," in *Proc. Design, Automation and Test in Europe (DATE)*, 2020.

[21] Z. Shi, H. Pan, C. Hu, and Z. Xie, "DeepGate2: Functionality-Aware Circuit Representation Learning," in *Proc. 42nd IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2023.

[22] L. Alrahis, S. Patnaik, M. Shafique, and O. Sinanoglu, "UNTANGLE: Unlocking Routing and Logic Obfuscation Using Graph Neural Networks-based Link Prediction," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2021.

[23] Y. Ye and Z. Wang, "FPGNN-ATPG: An Efficient Fault Parallel Automatic Test Pattern Generator," in *Proc. Great Lakes Symp. on VLSI (GLSVLSI)*, 2023.

[24] Y. Zhang *et al.*, "DiLA: Enhancing LLM Tool Learning with Differential Logic Layer," arXiv:2402.11903, 2024.

[25] J. Wang et al., "GTAC: A Generative Transformer for Approximate Circuits," arXiv:2601.19906, 2026.

[26] N. Cingillioglu and A. Russo, "Differentiable Logic Machines," arXiv:2208.08310, 2022.

[27] K. Guo, S. Zeng, J. Yu, Y. Wang, and H. Yang, "[DL] A Survey of FPGA-Based Neural Network Inference Accelerators," *ACM Transactions on Reconfigurable Technology and Systems (TRETS)*, vol. 12, no. 1, pp. 1–26, 2019.

[28] C. Zhao, J. Wang, X. Jiang, J. Lou, and Y. Lin, "GTA: GPU-Accelerated Track Assignment with Lightweight Lookup Table for Conflict Detection," in *Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD)*, 2025.